Lecture 20 Practice Questions and Solutions

Question 1 (q1_sol.py)

You're building a puzzle game for little kids. Each round shows the player a word with its first and last letters swapped, and they have to work out the real word. The main code has already been written for you: see below.

However, it doesn't work because the scramble function is not defined. Define the scramble function that returns (not prints!) the scrambled version of the word. To scramble a word, swap the first and last letters. If the word has fewer than 2 characters, return the word unchanged. For example, scramble("python") returns "nythop" and scramble("a") returns "a".

Don't change any of the given code: just add the scramble function.

Sample Run

```
Kid Scramble!

What is the real word for: ootatp? potato
Correct!

What is the real word for: tarroc? carot
Incorrect. The real word is carrot.

What is the real word for: whrot? Throw
Incorrect. The real word is throw.

What is the real word for: eppla? apple
Correct!

What is the real word for: aananb? banana
Correct!

You got 3 out of 5 correct.
```

Given Code

```
words = ["potato", "carrot", "throw", "apple", "banana"]

print("Kid Scramble!")

correct = 0
for w in words:
    scrambled = scramble(w)
    guess = input(f"
What is the real word for: {scrambled}? ").strip()
    if guess == w:
        print("Correct!")
        correct += 1
```

```
    else:
        print(f"Incorrect. The real word is {w}.")

print(f"
You got {correct} out of {len(words)} correct.")
```

Sample Solution

```
def scramble(word):
    if len(word) < 2:
        return word
    return word[-1] + word[1:-1] + word[0]

words = ["potato", "carrot", "throw", "apple", "banana"]

print("Kid Scramble!")

correct = 0
for w in words:
    scrambled = scramble(w)
    guess = input(f"\nWhat is the real word for: {scrambled}? ").strip()
    if guess == w:
        print("Correct!")
        correct += 1
    else:
        print(f"Incorrect. The real word is {w}.")

print(f"\nYou got {correct} out of {len(words)} correct.")
```

Question 2 (q2_sol.py)

A warehouse labels its boxes with product codes like "BX-4471". Sometimes the scanner misreads one character, and staff need to correct it. Write a function called fix_code(code, position, correct_char) that returns (not prints!) the corrected code.

Assume that code is always a string of at least 7 characters, that position is a valid index, and correct_char is a single character.

For example:

- fix_code("BX-4A71", 4, "9") returns "BX-4971"
- fix_code("TH-8328", 3, "7") returns "TH-7328"
- fix_code("XX-1111", 0, "R") returns "RX-1111"

Test your program by calling fix_code with the given examples and printing the results as shown.

Sample Run

```
BX-4A71 -> BX-4971
TH-8328 -> TH-7328
XX-1111 -> RX-1111
```

Sample Solution

```
def fix_code(code, position, correct_char):
    return code[:position] + correct_char + code[position+1:]

print(f"BX-4A71 -> {fix_code('BX-4A71', 4, '9')}")
print(f"TH-8328 -> {fix_code('TH-8328', 3, '7')}")
print(f"XX-1111 -> {fix_code('XX-1111', 0, 'R')}")
```

Question 3 (q3_sol.py)

The file trails.txt contains the names of some local hiking trails. The parks department wants to make trail signs all the same width, and so they need to know the longest trail name.

Write a program that opens trails.txt and then prints the longest trail name, plus its length. The length should include any spaces in the name, but not newlines or other whitespace characters.

Your program should work with any number of trails in the file.

Sample Run

```
Longest trail name: Baden-Powell Trail (18 characters)
```

Sample Solution

```
all_trails = open('trails.txt')
longest_trail = ''
for trail in all_trails:
    trail = trail.strip()
    if len(trail) > len(longest_trail):
        longest_trail = trail

print(f"Longest trail name: {longest_trail} ({len(longest_trail)} characters)")
```